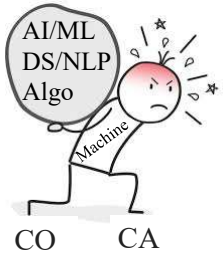
CS-2710-1 Spring 2026

Computer Organization and Systems (COS)



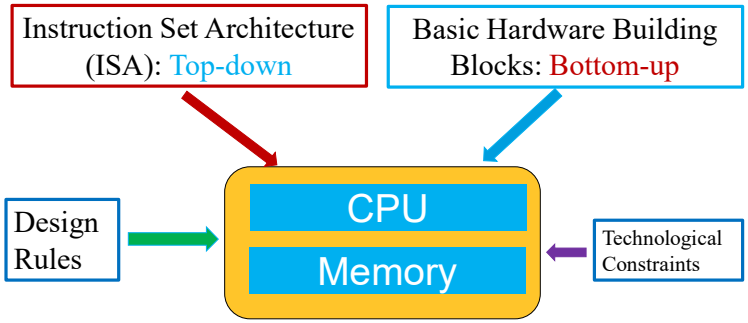
Lecture #14

25 March 2026

Ashoka University

BBB

How to design a processor?

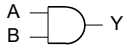


Processor (CPU) Design: Begin from ISA

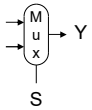
- We have studied CO-fundamentals, MIPS, and ALU design. Now, we're ready to implement the MIPS "core":
  - load-store instructions: `lw, sw`
  - reg-reg instructions: `add, sub, and, or, slt`
  - control flow instructions: `beq, j`

Basic Building Blocks

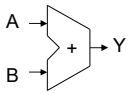
- **AND-gate**
  - **Y = A & B**



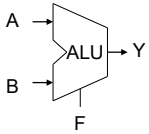
- Multiplexer



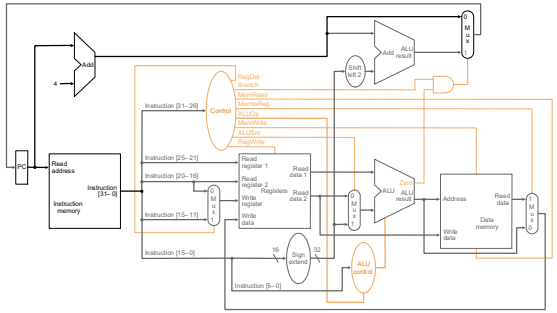
- Adder
  - Y = A + B



- Arithmetic/Logic Unit
  - Y = F(A, B)



Control Signal Requirement



Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

How to design the control unit?

- Input:
  --- Op-code and function-field bits from instructions
- Output:
  --- Desired control signals needed during the execution of an instruction
- For single-cycle implementation, the controller can be designed by a combinational circuit

Control

- **Selecting the operations to perform (ALU, read/write, etc.)**
- **Controlling the flow of data (multiplexor inputs)**
- **Information comes from the 32 bits of the instruction**
- **Example:**

add $8, $17, $18 **Instruction Format:**

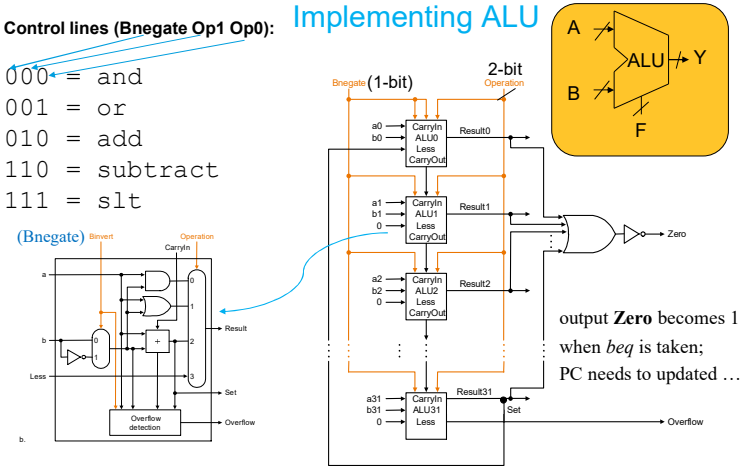
000000	10001	10010	01000	00000	100000
op	rs	rt	rd	shamt	funct

- **ALU's operation based on instruction type and function code**

Implementing ALU

Control lines (Bnegate Op1 Op0):

```
000 = and
001 = or
010 = add
110 = subtract
111 = slt
```

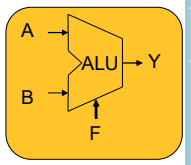


output **Zero** becomes 1 when *beq* is taken; PC needs to updated …

This ALU implements MIPS *add*, *sub*, *and*, *or*, *slt*, and *beq*

ALU Control

- ALU used for
  - Load/Store: F = add
  - Branch: F = subtract
  - R-type: F depends on funct field



ALU control (F)	Function
000	AND
001	OR
010	add
110	subtract
111	set-on-less-than

ALU Control

- Assume 2-bit ALUOp derived from opcode
  - Combinational logic derives ALU control

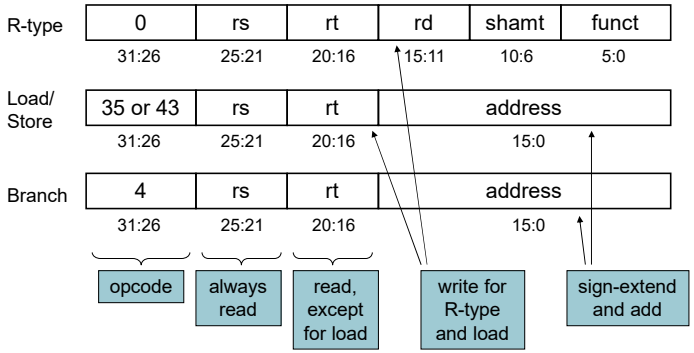
opcode	ALUOp	Operation	funct	ALU function	ALU control
lw	00	load word	XXXXXX	add	010
sw	00	store word	XXXXXX	add	010
beq	01	branch equal	XXXXXX	subtract	110
R-type	10	add	100000	add	010
		subtract	100010	subtract	110
		AND	100100	AND	000
		OR	100101	OR	001
		set-on-less-than	101010	set-on-less-than	111

Inputs to control logic

outputs of ALU control logic

The Main Control Unit

- Control signals derived from instruction



opcode	ALUOp	Operation	funct	ALU function	ALU control
lw	00	load word	XXXXXX	add	010
sw	00	store word	XXXXXX	add	010
beq	01	branch equal	XXXXXX	subtract	110
R-type	10	add	100000	add	010
		subtract	100010	subtract	110
		AND	100100	AND	000
		OR	100101	OR	001
		set-on-less-than	101010	set-on-less-than	111

Inputs to control logic

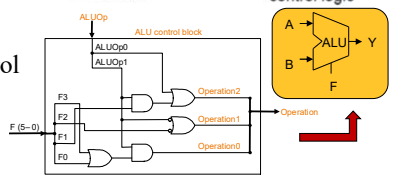
outputs of control logic

Control Block Design

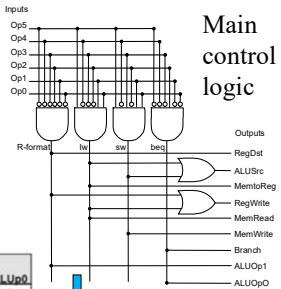
- Simple combinational logic
- Inputs: Op-code and Function-field of Instructions

Outputs: Desired control signals

ALU control logic

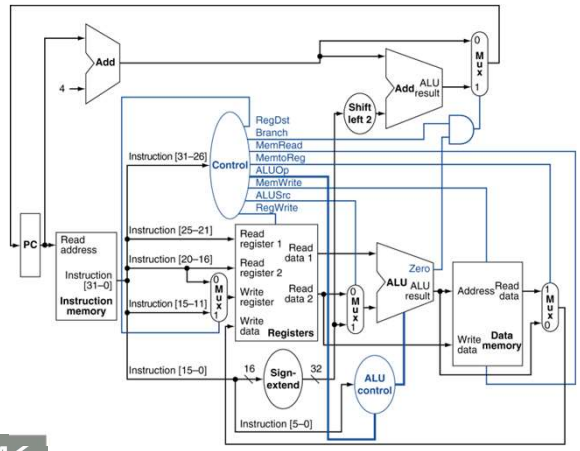


Main control logic

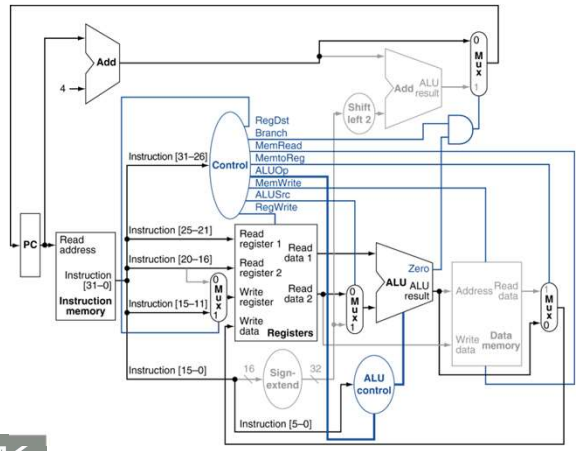


Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

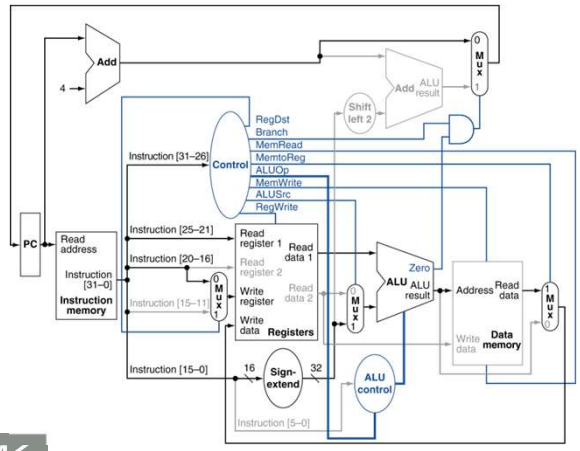
Datapath With Control



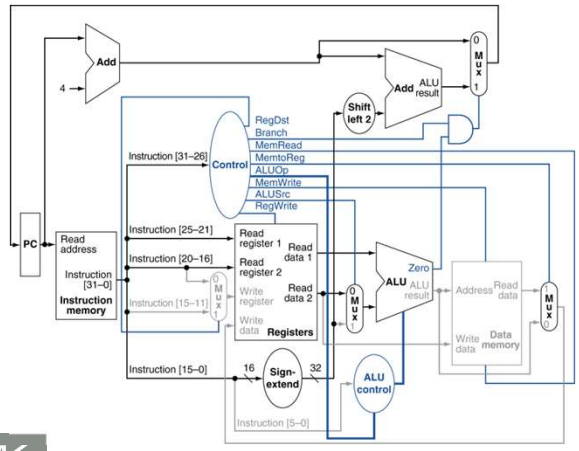
R-Type Instruction



Load Instruction



Branch-on-Equal Instruction

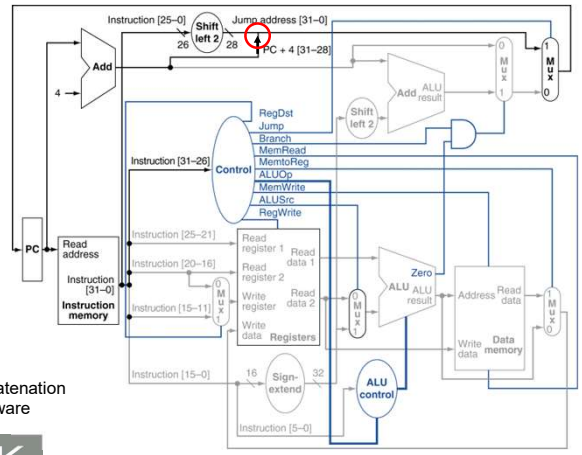


Implementing Jumps

Jump	2	address
	31:26	25:0

- Jump uses word address
- Update PC with concatenation of
  - Top 4 bits of old PC
  - 26-bit jump address
  - Least significant two bits: 00
- Need an extra control signal decoded from op-code

MIPS Processor for implementing
lw, sw, add, sub, addi, subi, and, or, slt, beq, j



concatenation hardware

Performance Issues

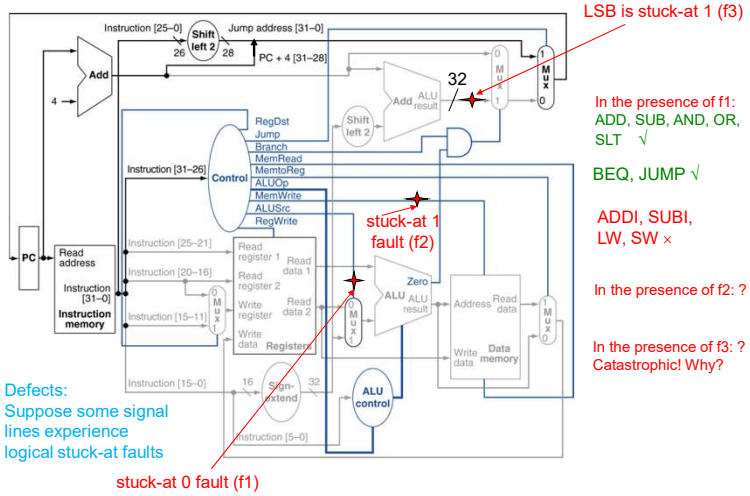
- Longest delay determines clock period
  - Critical path: load instruction
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle
  - Making the common case fast
- We will improve performance by pipelining

Key Points

- CPU is just a collection of state and combinational logic
- We have just designed a processor, at least in terms of functionality
- CPU time = IC * CPI * CCT
  - single-cycle machine has limited performance
  - Needs improved techniques such as pipelining

Logic Faults in MIPS Processor

In MIPS, programs typically start at memory address 0x00400000



In the presence of f1: ADD, SUB, AND, OR, SLT √

BEQ, JUMP √

ADDI, SUBI, LW, SW ×

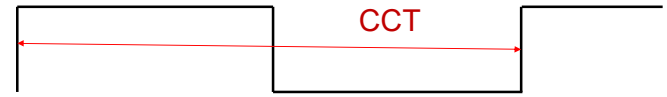
In the presence of f2: ?

In the presence of f3: ? Catastrophic! Why?

Defects:
Suppose some signal lines experience logical stuck-at faults

Processor Design

Single-Cycle Processor



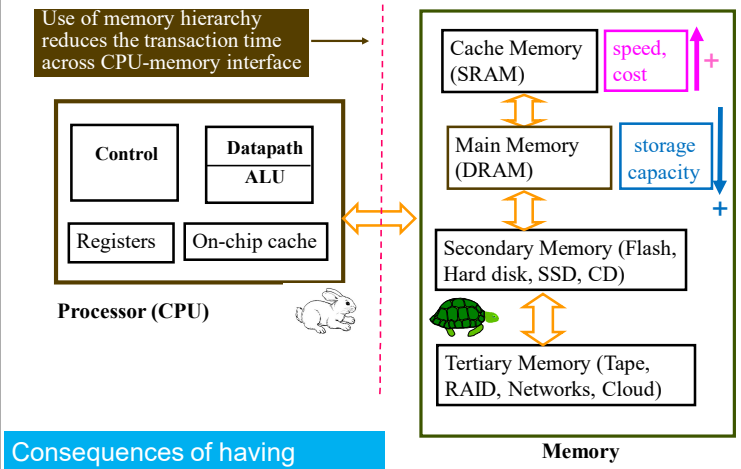
IF → ID/RR → EX → Mem → WB

Single-Cycle Processor Design

What are the design-characteristics/ consequences of having “single-cycle” based processor?

Complete View of Computer Architecture

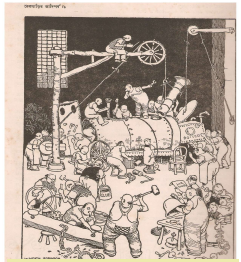
Memory transaction is inherently slow compared to logic



Consequences of having single-cycle processor design?

Description

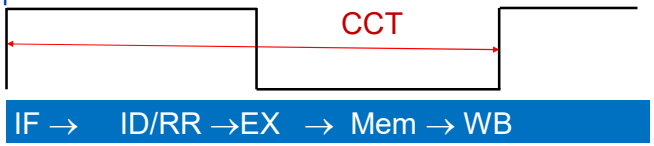
Job: Manufacture *n* cars or locomotive engines



Courtesy: Satyajit Ray

Naïve Approach: Engage all labor and machinery to finish building one car, and proceed to produce the next;

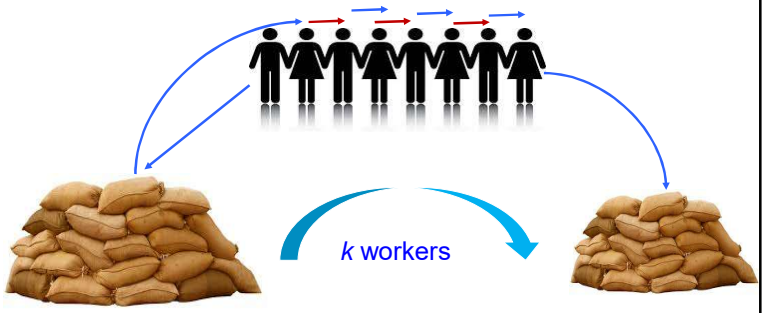
Single-Cycle Processor: Finish executing one instruction and proceed to the next …



The concept of *pipelining* is very old

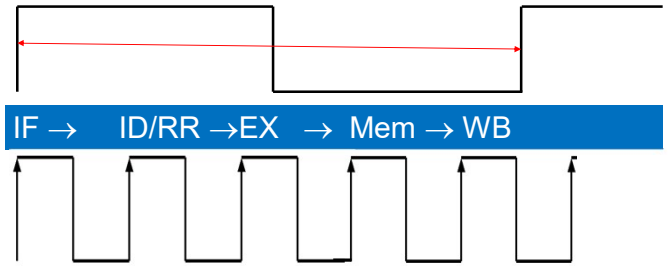
It had been in practice in industrial assembly-lines (e.g., automobiles) for a long time

Bag transportation via human chain



Processor Design

Single-Cycle Processor



Multi-Cycle Processor

Description: Assembly Line

Job: Manufacture *n* cars or locomotive engines

Pipelined Approach: Divide, Forward, and Conquer

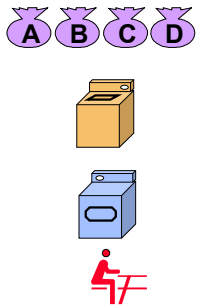
Multiple phases are needed to complete production;

Each phase needs specific expertise/ instruments/time

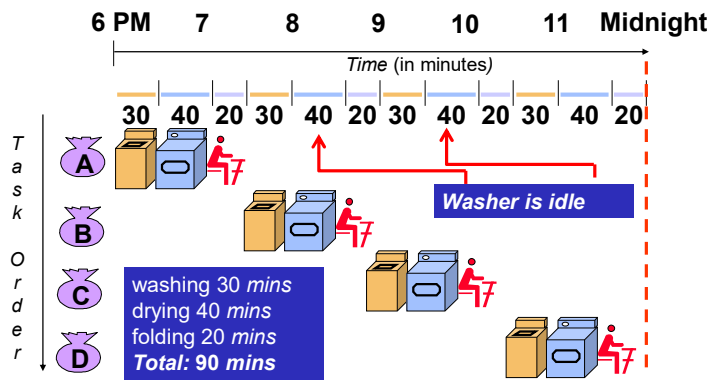
Goal: How to optimize the total completion time (cost) for producing *n* cars

What is Pipelining

- Laundry Room
- Four students A, B, C, D each has one load of clothes to wash, dry, and fold
- Washer takes 30 minutes
- Dryer takes 40 minutes
- Folder takes 20 minutes



What Is Pipelining: Laundry Example



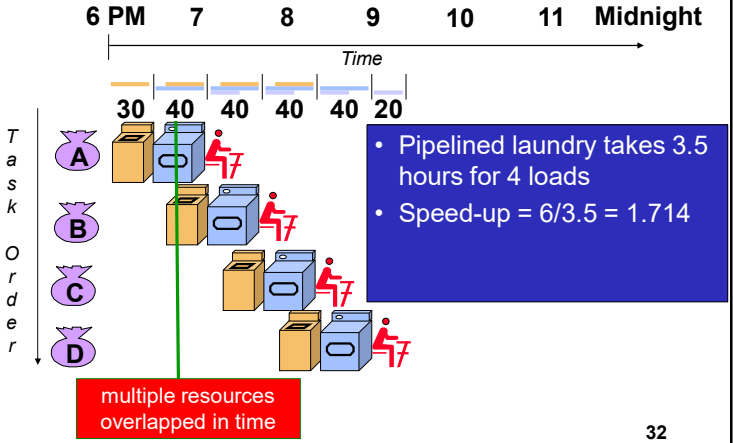
Sequential laundry takes 6 hours for four users; Work finishes at 00:00

If they learned pipelining, how long would laundry take?

Only one resource is busy at a time

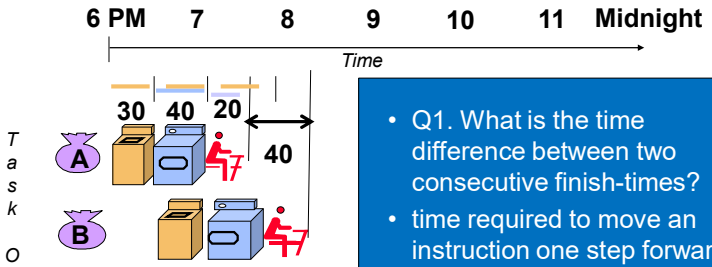
Basic Idea of Pipelining

Minimize idle-time for resources



- Pipelined laundry takes 3.5 hours for 4 loads
- Speed-up = 6/3.5 = 1.714

Impact of Pipeline Stages

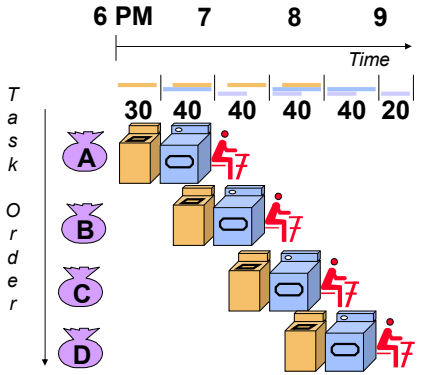


- Q1. What is the time difference between two consecutive finish-times?
- time required to move an instruction one step forward
- → longest time (slowest) among pipe-stages

Q2. What is the maximum speed-up compared to non-pipelined approach??
→ limited by the number of pipe-stages

- Speed-up when $n \to \infty$
  $\approx 90/40 = 2.25$
  < 3 (number of pipe-stages)

Pipelining Lessons



- Pipelining doesn't help **latency** of single task, it helps **throughput** of entire workload
- Pipeline rate limited by **slowest** pipeline stage
- **Multiple** tasks operating simultaneously
- Potential speed-up ≤ **Number pipe stages (pipe-depth)**
- Unbalanced lengths of pipe stages reduces speed-up
- Time to "**fill**" pipeline and time to "**drain**" it reduces speedup
- **Stall for Dependences**

Learned So Far ….

- Pipelining is widely used in modern processors.
- Pipelining improves system performance in terms of throughput.
- Pipelined organization requires sophisticated compilation techniques.